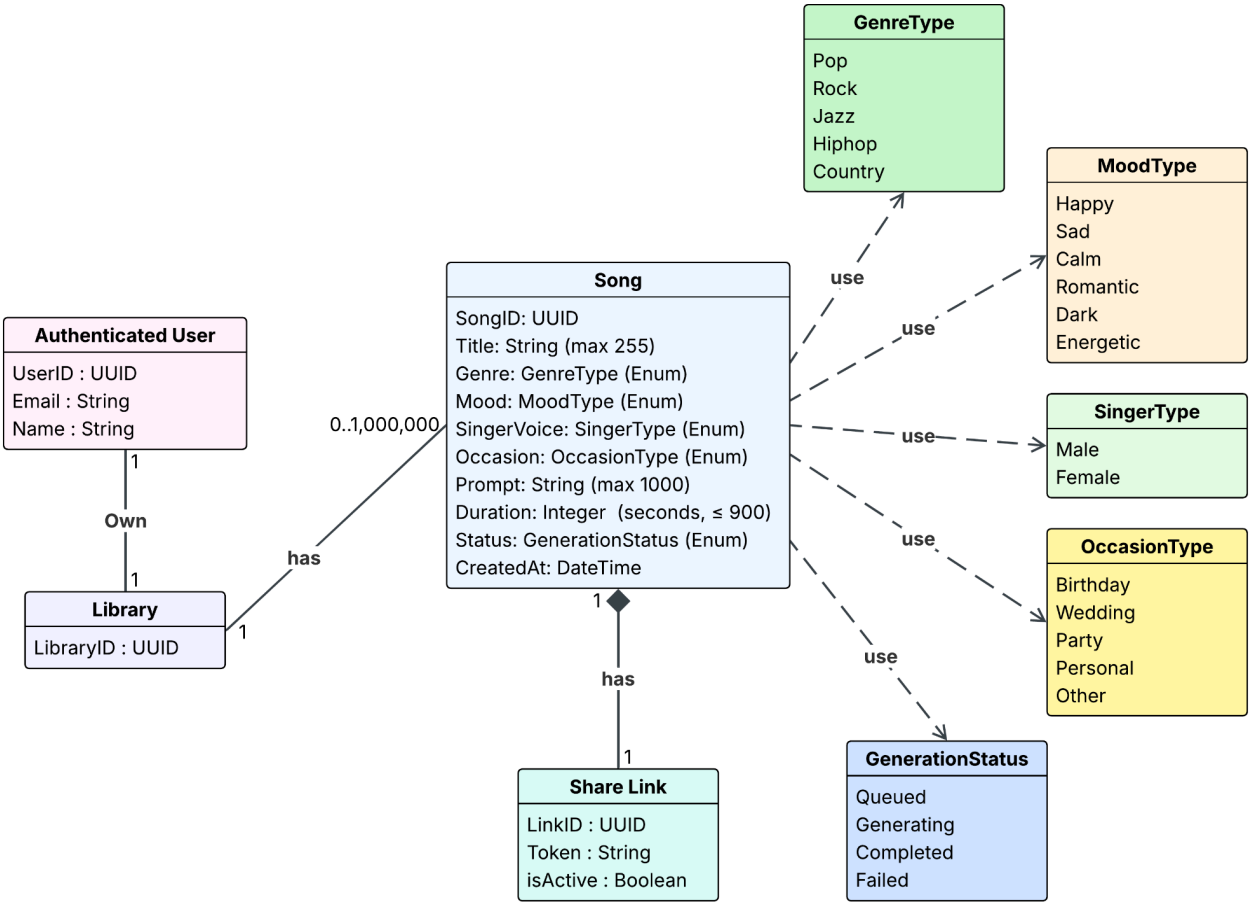
Domain model

By Peraya Leangsongchai



Supporting note for Cithara Domain Modeling

By Peraya Leangsongchai

1. Key Assumption

The following assumptions were made during system modeling and design:

1. All users must authenticate using Google OAuth before accessing any system feature.
2. Every authenticated user has equal permissions to generate, listen, share, download, and delete their own songs.
3. Each user owns exactly one personal library.
4. Each song automatically generates exactly one shareable link.
5. Shared songs can only be accessed by authenticated users who possess the valid link.
6. Song generation is performed using a third-party AI service (Suno AI).
7. The system supports modern browsers, specifically Google Chrome and Mozilla Firefox.
8. The system is deployed in a secure HTTPS environment (including localhost HTTPS during development).
9. The system is intended for non-technical users and prioritizes simplicity.
10. The system is free to use and does not include monetization features.
11. Single-Category Selection: To ensure a clean UI and structured data, users select exactly one primary value for Genre, Mood, Occasion, and Singer Voice from the provided lists
12. Fixed Enumerations: The system uses predefined sets for song attributes to ensure data consistency and prevent invalid user inputs
13. If a user requires a song to fit multiple categories (e.g., both a "Birthday" and a "Funeral"), they are assumed to include those specific details within the 1,000-character prompt text.

These assumptions help clarify system boundaries and remove ambiguity in requirements.

2. Constraints and business rules

User and Library Constraints

1. Each user shall have exactly one private library.
2. Libraries cannot be shared between users.
3. A library may contain a maximum of 1,000,000 songs.
4. Users may delete songs from their own library only.
5. There is no generation limit, provided the library capacity is not exceeded.

This is modeled as: User 1 - 1 Library and Library 1 - 0..1,000,000 Song

Song Generation Constraints

6. Users must be authenticated before generating songs.
7. Song duration must not exceed 15 minutes.
8. Prompt text must not exceed 1,000 characters.
9. Song generation status must be tracked (Queued, Generating, Completed, Failed).
10. Supported output formats include MP3, M4A, and WAV.

These constraints are reflected in the Song entity attributes and enumerations.

Sharing Constraints

11. Each song has exactly one shared link.
12. A share link cannot exist without its associated song.
13. When a song is deleted, its share link is automatically deleted.
14. Only authenticated users with a valid link may access a shared song.
15. Entire libraries cannot be shared.

This relationship is modeled using composition: Song 1 *- 1 ShareLink

This ensures lifecycle dependency and enforces the one song one link rule.

3. Traceability Mapping

The following table demonstrates alignment between functional requirements and the system design.

Requirement ID	Requirement Summary	Design Mapping
AUTH-1	Authenticate using Google Sign-In	Authentication is represented by the User domain entity, which relies on external integration with Google OAuth for identity verification. System access control constraints ensure that only authenticated User instances may interact with other domain entities.
AUTH-2	Restrict access to authenticated users only	Access control is enforced as a system-wide constraint. All domain interactions involving Library, Song, and ShareLink require an authenticated User instance.
AUTH-3	Prompt unauthenticated users to sign in	Preconditions in use cases enforce authentication prior to access. No domain entities are accessible without successful user authentication.
SONG-1	Provide multi-page form for song creation	The multi-step input process corresponds to a structured population of attributes within the Song entity prior to generation.
SONG-2	Allow input of title, genre, mood, occasion, and voice preference	These inputs are modeled as attributes of the Song entity. Controlled values (GenreType, MoodType, OccasionType, SingerType) are implemented as enumerations to enforce domain consistency.
SONG-3	Allow optional prompt text up to 1000 characters	The Song.prompt attribute stores extended contextual information. A length constraint (≤1000 characters) is defined as a domain rule.
SONG-4	Allow review of input before generation	The review step occurs prior to the creation of a finalized Song instance submitted for generation. Attribute validation ensures completeness before status transition.
SONG-5	Allow editing inputs before confirmation	The Song entity remains mutable prior to submission to the generation process. Attribute updates are permitted until generation begins.
GEN-1	Send structured parameters to AI service	The Song entity aggregates structured generation parameters which are transmitted to an external AI service (Suno AI). The AI service is modeled as an external dependency.

GEN-2	Display song generation status	The Song.status attribute, defined by the GenerationStatus enumeration (Queued, Generating, Completed, Failed), tracks lifecycle progression.
GEN-3	Support regeneration with modified inputs	Regeneration is modeled as modification of existing Song attributes followed by resubmission to the external generation service.
GEN-4	Limit generated song duration to 15 minutes	The Song.duration attribute includes a domain constraint (≤900 seconds).
GEN-5	Support MP3, M4A, and WAV formats	Audio output format is constrained by system rules and stored as part of the generated song metadata associated with the Song entity.
LIB-1	Automatically store generated songs in personal library	A one-to-many association exists between Library and Song (Library 1 — 0...1,000,000 Song). Songs are added to the owning user's Library upon successful generation.
LIB-2	Associate each song with a creation timestamp	The Song.createdAt attribute records the creation date and time.
LIB-3	Allow users to delete songs	Deletion of a Song instance removes it from the associated Library. Due to composition, its associated ShareLink is also deleted.
LIB-4	Display empty-state message when library has no songs	If a Library contains zero associated Song instances, the system presents an empty-state condition.
LIB-5	Each user has a single private library	A 1-to-1 association between User and Library enforces exclusive ownership.
PLAY-1	Provide in-browser playback controls	Playback functionality operates on generated audio files associated with the Song entity.
PLAY-2	Display song metadata during playback	Metadata is derived from attributes of the Song entity (title, genre, mood, createdAt).
PLAY-3	Allow playback of shared songs for authenticated users with valid links	Access to a Song via ShareLink requires authentication and valid link verification.
SHARE-1	Allow users to download their own songs	Ownership is enforced through the User -> Library ->Song relationship. Download permissions are restricted to the owning User.
SHARE-2	Generate shareable link for individual songs	Each Song is composed with exactly one ShareLink entity (composition relationship), ensuring lifecycle dependency.
SHARE-3	Restrict access to shared songs to authenticated users with link	Access requires both authentication and validation of the ShareLink.token.
SHARE-4	Prevent sharing of entire user libraries	No association exists between different Library instances, and sharing is restricted to individual Song objects only.

4. Design Decisions and Modeling Rationale

4.1 User Modeling Decision

The system defines a single **User** entity in the class diagram.

Although users may act as:

- Song Creator (when generating songs)
- Listener (when accessing shared songs)

These represent behavioral roles in the use case diagram rather than structural differences in the domain model.

All authenticated users share identical attributes and capabilities. Therefore, no subclassing (e.g., Creator or Listener) is used in the class diagram to avoid unnecessary inheritance and maintain a clean structural model.

4.2 Use of Enumerations

Enumerations are used to restrict attribute values to predefined sets, ensuring data consistency and preventing invalid inputs.

Examples:

- Genre = { Rock, Pop, Jazz, HipHop, Country }
- Mood = { Happy, Sad, Energetic, Calm }
- GenerationStatus = { Queued, Generating, Completed, Failed }

The use of enumerations improves clarity, reduces ambiguity, and strengthens system constraints.

By limiting the Song entity to a single enumeration value per category, the system maintains a consistent database schema while allowing flexibility through the Song.prompt attribute for complex, multi-genre descriptions

4.3 Separation of Behavioral and Structural Modeling

The system clearly distinguishes between:

- Use Case Diagram ⟶ describes user interactions and behavioral roles.
- Class Diagram ⟶ describes structural entities and data relationships.

Behavioral roles (Creator, Listener) are not modeled as subclasses because they do not represent distinct data structures.

4.4 Minimal Attribute Design

Only attributes explicitly required by system requirements are included in the domain model. Unnecessary attributes (e.g., user registration timestamps) are omitted unless required by functional or non-functional specifications. This ensures requirement-driven modeling and avoids over-engineering.

Conclusion

The system design fully satisfies the defined functional requirements. Structural entities in the class diagram support all required behaviors, while authentication, UI flows, and third-party integrations are appropriately modeled at the architectural and use case levels. Clear multiplicities, composition relationships, bounded constraints, and enumerations ensure precision, scalability, and requirement traceability.